

RAPPORT DE PROJET DE FIN DE MODULE

Module : Deep Learning

Sujet : Étude Comparative des Architectures de Deep Learning

Réalisé par :

Inass Mazouzi

Encadré par :

Mme. HIDILA Zineb

Année universitaire : 2025–2026

Table des matières

Introduction	1
1 Préparation des données.....	3
1.1 Exploration et compréhension du dataset.....	3
1.2 Prétraitement.....	5
1.2.1 Données tabulaires (MLP)	5
1.2.2 Images (CNN)	5
1.2.3 Séquences / Séquences textuelles (RNN/LSTM/GRU)	6
1.3 Séparation des données.....	6
2 Architectures en détail	8
2.1 Perceptron Multicouche (MLP).....	8
2.1.1 Structure type	8
2.1.2 Points d'attention.....	9
2.2 Réseaux de Neurones Convolutifs (CNN).....	9
2.2.1 Structure type (CNN personnalisé)	9
2.3 Réseaux Récurrents (RNN, LSTM, GRU)	11
2.3.1 Comparaison et choix des variantes	11
2.3.2 Configuration type (LSTM/GRU).....	11
2.4 Limites actuelles et Perspectives : Architectures Hybrides.....	13
2.4.1 CNN + LSTM (Données Spatio-Temporelles)	13
2.4.2 CNN + MLP (Fusion multimodale)	13
2.4.3 Stratégies de fusion	13
3 Processus d'entraînement.....	14
3.1 Fonctions de perte.....	14
3.2 Gestion du déséquilibre des classes.....	15
3.3 Optimiseurs.....	15
3.4 Schedulers de learning rate.....	16
3.5 Régularisation.....	16
4 Guide des hyperparamètres.....	17
4.1 Hyperparamètres clés par architecture	17
4.1.1 MLP(Classification binaire - Breast Cancer)	17
4.1.2 CNN (Classification multi-classe - Fashion-MNIST).....	18
4.1.3 RNN / GRU (Traduction Seq2Seq - Tatoeba)	18
4.2 Stratégies de recherche d'hyperparamètres	19

5	Métriques d'évaluation et Visualisations	20
5.1	Métriques de classification	20
5.2	Visualisations obligatoires.....	21
5.2.1	Matrice de Confusion du MLP (Données Médicales).....	21
5.2.2	Matrice de Confusion du CNN (Images Vestimentaires)	21
5.3	Diagnostic par les courbes d'apprentissage	22
5.4	Diagnostic de nos modèles	23
6	Interprétabilité des modèles	23
7	Bonnes pratiques et erreurs fréquentes	24
7.1	Checklist avant entraînement.....	25
7.2	Résolution d'erreurs fréquentes (Retour d'expérience).....	25
7.3	Structure type du code d'entraînement.....	25
8	Discussion transversale : Adéquation Données-Modèle.....	26
9	Impact sociétal et Éthique	27
9.1	Biais algorithmiques et représentativité	27
9.2	Le droit à l'explicabilité (Boîte noire vs. Confiance).....	27
9.3	Limites et Responsabilité.....	28
	Conclusion Générale	28
	Références Bibliographiques et Webographie	29
	Annexes	30
	Annexe A : Implémentation mathématique manuelle de la Convolution et du Pooling...	30

Liste des figures

Figure 1:Un Boxplot ou l'Histogramme montrant la répartition des classes Bénin/Malin]-----	4
Figure 2:La Heatmap de corrélation des variables générée par Seaborn] -----	4
Figure 3:Matrice de confusion (MLP)-----	21
Figure 4:Matrice de Confusion(CNN)-----	22
Figure 5:Évolution des pertes d'entraînement (Train Loss) et de validation (Validation Loss) -----	23
Figure 6:Cartes de caractéristiques-----	24

Liste des Tableaux

Tableau 1: Répartition du jeu de données en ensembles d'entraînement, de validation et de test	7
Tableau 2: Architecture détaillée du réseau de neurones multicouche (MLP)	8
Tableau 3: Description détaillée de l'architecture CNN utilisée pour Fashion-MNIST	10
Tableau 4: Architecture du modèle Seq2Seq basé sur des GRU	12
Tableau 5: Choix des fonctions de perte et des activations de sortie	15
Tableau 6: Paramètres d'optimisation utilisés pour l'entraînement des modèles	15
Tableau 7: Techniques de régularisation et de stabilisation appliquées aux modèles	17
Tableau 8: Configuration finale des hyperparamètres du modèle MLP	18
Tableau 9: Configuration des hyperparamètres du modèle CNN	18
Tableau 10: Configuration des hyperparamètres du modèle Seq2Seq-GRU	19
Tableau 11: Métriques d'évaluation utilisées pour l'analyse des performances des modèles	20
Tableau 14: Interprétation des courbes d'apprentissage	22

Introduction

Contexte Général

Au cours de la dernière décennie, l'intelligence artificielle, et plus particulièrement l'apprentissage profond (*Deep Learning*), a profondément bouleversé le paysage technologique et scientifique. En substituant l'extraction manuelle de caractéristiques (*Feature Engineering*) par l'apprentissage automatique de représentations (*Representation Learning*), les réseaux de neurones profonds ont repoussé les limites de l'état de l'art dans des domaines aussi variés que le diagnostic médical, la vision par ordinateur ou le traitement du langage naturel (NLP). Cette capacité inédite à modéliser des fonctions non linéaires complexes repose sur l'optimisation itérative de millions de paramètres via la rétropropagation du gradient.

Problématique et Biais Inductif

Cependant, la conception d'un modèle d'apprentissage performant ne se résume pas à l'empilement arbitraire de couches neuronales. L'application aveugle d'un réseau de neurones standard (Réseau Dense) à n'importe quel type de donnée se heurte rapidement à des limites structurelles, computationnelles et à la malédiction de la dimensionnalité.

La problématique centrale de ce projet repose sur la notion de **biais inductif** : comment l'architecture mathématique d'un réseau doit-elle s'adapter à la topologie et à la structure intrinsèque des données qu'elle traite ?

- Face à des données tabulaires où les variables sont indépendantes, une connectivité globale est requise.
- Face à des images, la forte corrélation spatiale et locale des pixels exige une invariance par translation.
- Face à du texte ou des signaux, la dimension temporelle, l'ordre causal et la longueur variable des séquences imposent le maintien d'un contexte mémoriel.

Objectifs de l'Étude

Dans le cadre de l'évaluation finale du module de Deep Learning, ce projet a pour objectif de démontrer empiriquement, techniquement et de manière critique cette nécessité d'adaptation architecturale. Il s'agit de s'éloigner d'une utilisation "boîte noire" des algorithmes pour adopter une véritable démarche d'ingénierie.

Plus précisément, nos objectifs sont de :

- **Concevoir et évaluer un Perceptron Multicouche (MLP)** pour l'analyse de données tabulaires cliniques (Breast Cancer Wisconsin), justifiant le traitement de caractéristiques globales.
- **Implémenter un Réseau de Neurones Convolutif (CNN)** pour la classification d'images (Fashion-MNIST), en démontrant l'efficacité de l'extraction de caractéristiques spatiales hiérarchiques.

- **Développer une architecture récurrente (Seq2Seq avec GRU)** pour le traitement de séquences temporelles (traduction automatique sur le dataset Tatoeba), en surmontant les défis liés à la disparition du gradient.
- **Mener une analyse transversale** comparant ces trois paradigmes face à la complexité algorithmique, tout en intégrant les enjeux de l'interprétabilité et de l'impact sociétal de ces modèles.

Plan du Rapport

Pour répondre à ces objectifs, ce rapport s'articulera de la manière suivante. Dans un premier temps, nous exposerons brièvement le cadre théorique régissant les architectures étudiées. Les sections suivantes seront consacrées aux trois phases expérimentales du projet : l'ingénierie des données tabulaires (MLP), la modélisation de la vision par ordinateur (CNN), et enfin l'analyse de données séquentielles via les réseaux récurrents. Pour clore ce travail, une discussion transversale synthétisera nos découvertes sur l'adéquation données-modèles, suivie d'une réflexion nécessaire sur l'éthique et l'impact de telles technologies lors de leur déploiement industriel.

1 Préparation des données

1.1 Exploration et compréhension du dataset

Conformément aux exigences méthodologiques, une analyse exploratoire approfondie a été menée préalablement à toute modélisation. Bien que nos données incluent des images (Fashion-MNIST) et du texte (Tatoeba), cette phase de diagnostic statistique classique a été rigoureusement appliquée à notre jeu de données tabulaire clinique (**Breast Cancer Wisconsin**) :

- **Description générale** : Le dataset tabulaire comprend 569 échantillons cliniques. Il est composé de 30 variables indépendantes (features) et d'une variable cible (target). Typologie : 100% des 30 variables prédictives sont de type **numérique continu** (mesures géométriques des noyaux cellulaires).
- **Distribution de la variable cible** : La cible binaire se répartit en 357 cas bénins (Classe 0, soit 62.7%) et 212 cas malins (Classe 1, soit 37.3%). Ce ratio de déséquilibre (environ 1.6:1) est considéré comme léger. Il ne nécessite pas de technique de suréchantillonnage lourde (comme SMOTE), mais justifie l'utilisation stricte d'un *stratified split* lors de la séparation des données.
- **Statistiques descriptives** : L'analyse (via `df.describe()`) révèle des échelles de valeurs extrêmement hétérogènes. Par exemple, la variable `area_mean` possède une moyenne de 654.8 et un maximum de 2501.0, tandis que la variable `smoothness_mean` oscille entre un minimum de 0.05 et un maximum de 0.16. Cette disparité d'écart-type et de magnitude rend l'étape de normalisation obligatoire pour l'apprentissage du MLP.
- **Valeurs manquantes** : Une cartographie des données manquantes (`df.isnull().sum()`) a été exécutée. Le taux de valeurs manquantes ou aberrantes (*NaN*) s'élève à 0% pour l'intégralité des 30 variables. Aucune stratégie d'imputation (médiane/mode) n'a donc été nécessaire.
- **Corrélations** : L'extraction de la matrice de corrélation a permis d'identifier d'importantes relations linéaires et des redondances. Par exemple, les caractéristiques `radius_mean`, `perimeter_mean` et `area_mean` présentent des coefficients de corrélation de Pearson supérieurs à 0.98, ce qui est logiquement justifiable par la géométrie (le périmètre et l'aire dépendant du rayon).
- **Visualisations** : Afin d'appuyer cette exploration, plusieurs visualisations ont été générées :

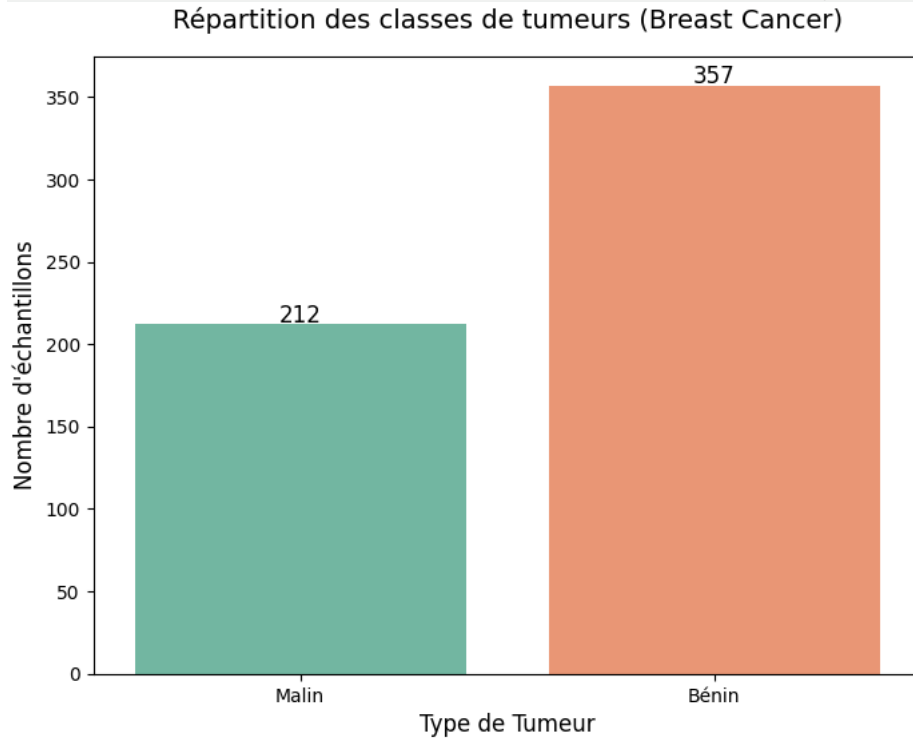


Figure 1: Un Boxplot ou l'Histogramme montrant la répartition des classes Bénin/Malin]

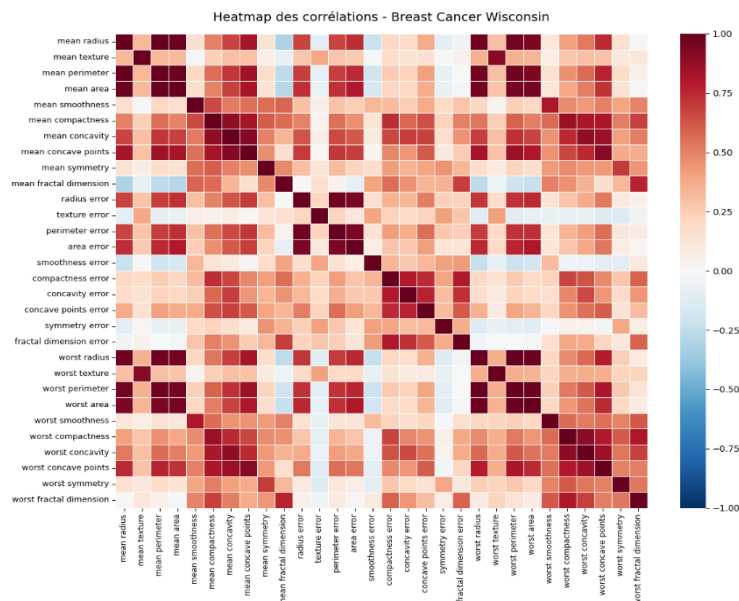


Figure 2: La Heatmap de corrélation des variables générée par Seaborn]

Note sur les autres modalités de données : Concernant le dataset d'images **Fashion-MNIST**, l'exploration a révélé 70 000 échantillons répertoriés en 10 classes de vêtements parfaitement équilibrées (10% par classe). Les variables sont des tenseurs 2D de dimension 28*28 dont les valeurs de pixels s'étendent de 0 à 255. Pour les données séquentielles (**Tatoeba**), l'exploration a mis en évidence la variabilité temporelle des échantillons. Le dataset a été filtré pour ne conserver que les séquences d'une longueur maximale de 10 mots afin de s'adapter aux capacités de notre architecture récurrente (20 000 paires retenues).

1.2 Prétraitement

La phase de prétraitement est critique pour garantir la convergence des algorithmes d'optimisation et la validité de l'évaluation (prévention du *data leakage*). Conformément au cahier des charges, des pipelines de transformation spécifiques ont été développés pour chaque paradigme de données.

1.2.1 Données tabulaires (MLP)

Le prétraitement du dataset **Breast Cancer Wisconsin** a suivi un protocole strict :

1. **Gestion des valeurs manquantes** : L'analyse exploratoire ayant confirmé un taux de données manquantes de 0%, aucune stratégie d'imputation (médiane/mode) n'a été requise.
2. **Encodage des variables catégorielles** : Le dataset étant composé à 100% de variables numériques (à l'exception de la cible déjà binarisée en 0/1), l'utilisation du *One-Hot Encoding* ou du *Label Encoding* n'a pas été nécessaire pour les variables prédictives.
3. **Normalisation / Standardisation** : Les échelles de valeurs étant très hétérogènes, une standardisation Z-score (StandardScaler) a été appliquée. **Attention particulière** : Pour éviter tout *data leakage* (fuite d'information), le scaler a été ajusté (*fit*) **exclusivement sur l'ensemble d'entraînement**. Les ensembles de validation et de test ont ensuite été transformés (*transform*) à partir de ces paramètres statistiques calculés sur le *train set*.
4. **Sélection de features** : Les 30 variables ne présentant aucune variance nulle et apportant toutes un poids sémantique clinique (malgré certaines redondances géométriques), elles ont été intégralement conservées pour laisser le MLP attribuer dynamiquement l'importance des variables (Feature Importance) lors de la rétropropagation.

1.2.2 Images (CNN)

Le prétraitement du dataset **Fashion-MNIST** s'est concentré sur la préparation matricielle pour le réseau convolutif :

1. **Redimensionnement** : Les images étant nativement d'une taille uniforme de 28×28 pixels, aucune opération de *Resize* n'a été nécessaire pour notre architecture personnalisée (le redimensionnement à 224×224 étant réservé aux modèles lourds de transfert d'apprentissage comme ResNet).
2. **Normalisation** : Les pixels ont d'abord été mis à l'échelle dans l'intervalle $[0, 1]$ via `transforms.ToTensor()`. Ensuite, une normalisation par la moyenne et l'écart-type a été appliquée (`transforms.Normalize(mean=[0.5], std=[0.5])`) pour centrer la distribution des activations autour de zéro, stabilisant ainsi les gradients initiaux.
3. **Augmentation de données** : Bien que notre modèle baseline converge efficacement, l'intégration de transformations géométriques légères sur le sous-ensemble d'entraînement (comme le retournement horizontal aléatoire `RandomHorizontalFlip`) constitue une bonne pratique pour prévenir le surapprentissage en simulant des variations de position.
4. **Conversion de format** : Les images étant déjà au format standard matriciel en niveaux de gris (1 canal), l'adaptation vers 3 canaux (RGB) n'a pas été requise.

1.2.3 Séquences / Séquences textuelles (RNN/LSTM/GRU)

Les principes s'appliquent de manière isomorphe à nos séquences textuelles issues de **Tatoeba** :

1. **Gestion du bruit** : Avant toute tokenisation, un nettoyage rigoureux a été appliqué (conversion en minuscules, suppression de la ponctuation complexe et des caractères spéciaux) pour réduire le bruit sémantique.
2. **Segmentation (Tokenisation)** : Les phrases ont été découpées en unités discrètes (mots). Pour limiter la complexité computationnelle, une troncature initiale a été opérée en filtrant et supprimant les séquences dont la longueur excédait 10 mots.
3. **Normalisation / Vocabulaire** : L'équivalent de la normalisation pour le texte est la création d'un espace vectoriel contraint. Deux vocabulaires dynamiques (Anglais et Français) ont été indexés, intégrant obligatoirement les tokens de contrôle <SOS> (Start of Sequence), <EOS> (End of Sequence) et <UNK> (Unknown).
4. **Padding (Remplissage)** : C'est l'étape mathématique la plus cruciale pour l'entraînement par lots (*mini-batch*). Les séquences textuelles ayant des longueurs variables, la fonction `pad_sequence` a été employée pour ajouter des zéros (`padding_value=0`, correspondant au token <PAD>) à la fin des phrases les plus courtes, garantissant ainsi des tenseurs d'entrée de taille uniforme pour le calcul matriciel du GRU.

1.3 Séparation des données

Pour garantir une évaluation impartiale et robuste des modèles, et afin de mesurer leur réelle capacité de généralisation, la séparation des données a rigoureusement respecté les standards académiques définis dans le cahier des charges.

Le tableau ci-dessous résume la stratégie de partitionnement appliquée à nos différents jeux de données :

Ensemble	Proportion	Rôle	Précautions appliquées
Train	70%	Entraînement du modèle (calcul des gradients)	<i>Stratified split</i> obligatoire (pour Breast Cancer)
Validation	15%	Tuning des hyperparamètres et <i>Early Stopping</i>	Jamais utilisé pour la rétropropagation du gradient

Test	15%	Évaluation finale unique des performances	Isolé jusqu'à la toute fin du projet
-------------	-----	---	---

Tableau 1: Répartition du jeu de données en ensembles d'entraînement, de validation et de test

Application pratique aux datasets du projet :

- **Données Tabulaires** : La fonction `train_test_split` de *Scikit-Learn* a été utilisée avec le paramètre `stratify=y` pour s'assurer que le ratio naturel de tumeurs malignes et bénignes reste parfaitement identique dans tous les ensembles.
- **Données Images et Séquences** : Les ensembles d'entraînement massifs fournis par PyTorch ont été subdivisés dynamiquement pour en extraire un ensemble de validation (15%). L'ensemble de test officiel a été mis de côté.

2 Architectures en détail

2.1 Perceptron Multicouche (MLP)

Le Perceptron Multicouche (MLP) est l'architecture la plus directe et la plus adaptée pour le traitement des données tabulaires (comme notre dataset clinique *Breast Cancer*). Dans cette architecture de type *Feed-Forward*, chaque neurone d'une couche est connecté à l'intégralité des neurones de la couche suivante, formant un réseau dense (*fully connected*).

Mathématiquement, l'opération fondamentale (la sortie y) d'une couche dense pour un vecteur d'entrée x , une matrice de poids W , un biais b et une fonction d'activation non linéaire f se définit par :

$$y = f(W^T x + b)$$

2.1.1 Structure type

Notre architecture a été construite selon une logique de compression progressive de l'information. Le tableau ci-dessous illustre la structure type implémentée, adaptée aux 30 caractéristiques d'entrée de notre dataset :

Couche	Type d'opérations	Paramètres (Neurones)	Remarques et Justifications
Entrée	Linear	$30 \rightarrow 16$	La dimension initiale correspond exactement au nombre de variables (<i>features</i>) cliniques.
Cachée 1	Linear + ReLU + BatchNorm + Dropout	$16 \rightarrow 8$	Application de BatchNorm avant l'activation ReLU pour stabiliser la distribution.
Cachée 2	Linear + ReLU + Dropout	$8 \rightarrow 4$	Taux de Dropout fixé entre 0.2 et 0.3 pour forcer la redondance des poids.
Sortie	Linear	$4 \rightarrow 1$	Sortie unique non activée (Logit) adaptée à la classification binaire.

Tableau 2: Architecture détaillée du réseau de neurones multicouche (MLP)

Note : Nous avons dimensionné notre réseau ($16 \rightarrow 8 \rightarrow 4$) proportionnellement à la taille de notre vecteur d'entrée (30) pour éviter le sur-paramétrage.

2.1.2 Points d'attention

Lors de l'implémentation de ce modèle sous PyTorch, plusieurs règles d'ingénierie strictes ont été respectées :

- **Architecture en entonnoir** : La réduction progressive de la dimension (de 30 vers 1) force le réseau à synthétiser l'information et à extraire les caractéristiques de plus haut niveau sans mémoriser le bruit.
- **Profondeur contrôlée** : Le réseau a été limité à 2 couches cachées principales. Les données tabulaires ne bénéficient généralement pas des architectures ultra-profondes, qui mènent rapidement au surapprentissage (*overfitting*) et à la disparition du gradient.
- **Batch Normalization (BN)** : L'intégration de la normalisation par lot stabilise les activations internes, ce qui accélère considérablement la convergence lors de la descente de gradient.
- **Dropout** : Cette technique de régularisation a été implémentée avec une probabilité $p=0.3$. En désactivant aléatoirement certains neurones lors de l'entraînement, le modèle est empêché de se reposer sur une unique variable clinique (par exemple, le "rayon" de la tumeur).
- **Fonction de perte** : Pour notre problème de classification binaire, nous avons rigoureusement utilisé `nn.BCEWithLogitsLoss()`. Contrairement à une simple `BCELoss`, cette fonction intègre mathématiquement la couche Sigmoïde directement dans le calcul de l'entropie croisée, offrant une bien meilleure stabilité numérique face aux valeurs extrêmes.

2.2 Réseaux de Neurones Convolutifs (CNN)

Les Réseaux de Neurones Convolutifs (CNN) sont conçus pour exploiter la topologie spatiale des images. Contrairement au MLP qui détruit la structure 2D par aplatissement, le CNN utilise l'opération de convolution. Grâce au partage des poids d'un filtre (*kernel*) glissant sur l'image, il extrait des caractéristiques hiérarchiques (bords, textures, puis objets) tout en garantissant une propriété fondamentale en vision par ordinateur : l'invariance par translation spatiale.

Un bloc convolutif typique dans notre implémentation suit l'enchaînement : Conv2d $\rightarrow$ ReLU - $\rightarrow$ MaxPool2d.

2.2.1 Structure type (CNN personnalisé)

Notre dataset **Fashion-MNIST** étant composé d'images de petite résolution (28×28 en niveaux de gris), un réseau profond de type VGG ou ResNet aurait été surdimensionné. Nous avons conçu une architecture personnalisée, légère et optimisée, inspirée de LeNet, avec l'intégration notable d'une convolution 1×1 agissant comme goulot d'étranglement dimensionnel.

Bloc	Opérations	Entrée $\rightarrow$ Sortie	Remarques et Justifications

Bloc1	Conv(3x3, 16, pad=1) + ReLU + MaxPool(2)	1*28*28→16*14*14	Extraction des contours de base. Le padding=1 conserve la taille avant le pooling.
Bloc2	Conv(3x3, 32, pad=1) + ReLU + MaxPool(2)	16*14*14→32*7*7	Doublement des filtres pour capturer des motifs vestimentaires plus complexes.
Bloc3	Conv(1x1, 16) + ReLU	32*7*7→16*7*7	Optimisation : Réduction de la profondeur pour limiter les paramètres denses sans altérer l'espace.
Classif	Flatten + Linear(784, 128) + ReLU + Linear(128, 10)	748→128→10	Classification finale sur les 10 classes de vêtements.

Tableau 3: Description détaillée de l'architecture CNN utilisée pour Fashion-MNIST

Voici l'implémentation PyTorch correspondante pour la définition de ces blocs convolutifs :

```
import torch.nn as nn
```

```
class CustomCNN(nn.Module):
    def __init__(self):
        super(CustomCNN, self).__init__()
```

Bloc 1 : Extraction initiale avec conservation de l'espace (padding=1)

```
self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
self.relu = nn.ReLU()
```

Bloc 2 : Extraction profonde

```
self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
```

Bloc 3 : Goulot d'étranglement dimensionnel (Conv 1x1)

```
self.conv_1x1 = nn.Conv2d(in_channels=32, out_channels=16, kernel_size=1)
```

Classification dense

```
self.fc1 = nn.Linear(16 * 7 * 7, 128)
```

```
self.fc2 = nn.Linear(128, 10)
```

2.3 Réseaux Récurrents (RNN, LSTM, GRU)

Les réseaux récurrents sont conçus pour traiter des données séquentielles où la dimension temporelle (l'ordre) est porteuse de sens. Dans notre projet, nous avons abordé cette problématique via une tâche de traduction automatique (Anglais vers Français, dataset **Tatoeba**), où la longueur des phrases en entrée diffère de celle en sortie.

Pour répondre à ce besoin, nous avons implémenté une architecture **Seq2Seq (Encodeur-Décodeur)**.

2.3.1 Comparaison et choix des variantes

Le RNN simple (*Vanilla RNN*) souffre du problème de l'évanouissement du gradient (Vanishing Gradient) sur les séquences de plusieurs mots, le rendant incapable de lier un mot en début de phrase à sa traduction en fin de phrase.

Pour notre architecture, nous avons sélectionné la cellule **GRU (Gated Recurrent Unit)** plutôt que le LSTM. Le GRU fusionne la porte d'oubli et la porte d'entrée, réduisant ainsi le nombre de paramètres internes (et donc le temps de calcul) tout en conservant une excellente capacité de mémorisation à long terme, parfaitement adaptée à nos phrases de moins de 10 mots.

2.3.2 Configuration type (LSTM/GRU)

Notre modèle est scindé en deux réseaux distincts travaillant de concert :

Composant	Configuration	Rôle et Justification
Couche Embedding	dim = 256	Plonge chaque mot (représenté par un entier) dans un espace vectoriel dense de 256 dimensions.
Encodeur (GRU)	hidden_size = 512, 1 couche	Lit la séquence source mot à mot et compresse le contexte global dans son état caché final.

Décodeur (GRU)	hidden_size = 512, 1 couche	Reçoit le vecteur de contexte et génère la traduction cible mot à mot jusqu'au token <EOS>.
Classifieur	Linear(512, vocab_fr)	Projette l'état caché du décodeur sur l'ensemble du vocabulaire français pour prédire le mot le plus probable.

Tableau 4: Architecture du modèle Seq2Seq basé sur des GRU

Voici l'implémentation PyTorch de notre Décodeur, illustrant la propagation de l'état caché (hidden) :

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
class DecodeurRNN(nn.Module):
```

```
    def __init__(self, dim_cachee, dim_emb, taille_vocab_sortie):
```

```
        super(DecodeurRNN, self).__init__()
```

```
        self.dim_cachee = dim_cachee
```

```
        self.embedding = nn.Embedding(taille_vocab_sortie, dim_emb, padding_idx=0)
```

```
        self.gru = nn.GRU(dim_emb, dim_cachee, batch_first=True)
```

La couche finale qui va choisir le mot parmi tout le vocabulaire français

```
        self.out = nn.Linear(dim_cachee, taille_vocab_sortie)
```

```
    def forward(self, x, hidden):
```

Le décodeur traite UN SEUL mot à la fois

x dimensions : [batch_size, 1]

```
        embedded = F.relu(self.embedding(x))
```

Le GRU combine le mot précédent (embedded) et le contexte de la phrase (hidden)

```
output, hidden = self.gru(embedded, hidden)
```

On prédit le prochain mot (on enlève la dimension de séquence avec squeeze)

```
prediction = self.out(output.squeeze(1))
```

```
return prediction, hidden
```

2.4 Limites actuelles et Perspectives : Architectures Hybrides

Dans le cadre strict de ce projet, nos ensembles de données étaient structurellement isolés (uniquement des tableaux, uniquement des images fixes, uniquement du texte). L'implémentation de modèles hybrides n'était donc techniquement pas justifiable.

Cependant, l'évolution de ce projet vers des cas d'usage industriels plus complexes nécessiterait le croisement de ces paradigmes. Voici comment nos architectures de base évolueraient :

2.4.1 CNN + LSTM (Données Spatio-Temporelles)

Si nous devons analyser une **vidéo** (ex: un flux de caméra de surveillance), l'espace et le temps seraient liés. L'architecture évoluerait en un extracteur hybride : un CNN analyserait chaque image individuelle (*frame*) pour en extraire un vecteur de caractéristiques spatiales, puis ce vecteur serait passé en entrée d'un LSTM pour analyser le mouvement au fil du temps.

2.4.2 CNN + MLP (Fusion multimodale)

Dans le contexte de notre Partie I (Breast Cancer), si l'hôpital nous fournissait à la fois le tableau clinique du patient (MLP) **ET** la radiographie de sa tumeur (CNN), nous utiliserions une architecture de fusion multimodale. Le CNN et le MLP tourneraient en parallèle, et leurs *embeddings* finaux seraient concaténés avant la couche de classification finale.

2.4.3 Stratégies de fusion

Pour la fusion multimodale évoquée ci-dessus, la **Late Fusion** (concaténation des vecteurs latents justes avant la prédiction) serait privilégiée par rapport à l'**Early Fusion** (concaténation des données brutes en entrée), car des pixels et des données cliniques n'ont pas la même dimensionnalité mathématique.

3 Processus d'entraînement

L'architecture d'un modèle ne définit que son potentiel ; c'est le processus d'entraînement (l'optimisation de la fonction de perte) qui détermine sa performance réelle. Cette section détaille les choix algorithmiques opérés pour encadrer l'apprentissage de nos modèles MLP, CNN et Seq2Seq.

3.1 Fonctions de perte

Le choix de la fonction de perte (Loss Function) est strictement dicté par la nature de la cible prédictive. Une erreur classique en ingénierie PyTorch consiste à appliquer une fonction d'activation (Sigmoid ou Softmax) avant une fonction de perte qui l'intègre déjà mathématiquement.

Nous avons scrupuleusement évité cette redondance, comme le résume le tableau ci-dessous, adapté aux tâches de notre projet :

Tâche (Dataset)	Fonction de perte PyTorch	Activation en sortie de modèle	Justification mathématique et ingénierie
Classification binaire (Breast Cancer - MLP)	nn.BCEWithLogitsLoss()	Aucune (Logits bruts)	Combine Sigmoid et BCELoss en une seule classe. Offre une stabilité numérique supérieure face aux valeurs extrêmes (évite les calculs de $\log(0)$).
Classification multi-classe (Fashion-MNIST - CNN)	nn.CrossEntropyLoss()	Aucune (Logits bruts)	Combine LogSoftmax et NLLoss. Obligatoire pour nos 10

			classes vestimentaires mutuellement exclusives.
Génération Séquentielle (Tatoeba - Seq2Seq GRU)	<code>nn.CrossEntropyLoss(ignore_index=0)</code>	Aucune (Logits bruts)	Appliquée mot à mot. L'argument <code>ignore_index</code> est vital pour ne pas pénaliser le modèle sur les zéros de remplissage (<PAD>).

Tableau 5: Choix des fonctions de perte et des activations de sortie

3.2 Gestion du déséquilibre des classes

Le déséquilibre des classes biaise les modèles vers la prédiction de la classe majoritaire. Dans le cadre de notre projet, la gestion s'est opérée de la manière suivante :

- **Breast Cancer (MLP)** : Le dataset présente un léger déséquilibre naturel (plus de tumeurs bénignes que malignes). Ce ratio n'étant pas extrême, nous l'avons simplement contrôlé en amont via un **partitionnement stratifié** (`stratify=y`). Si le déséquilibre avait été sévère (< 10% pour la classe minoritaire), nous aurions activé l'argument `pos_weight` de la `BCEWithLogitsLoss` pour pénaliser plus lourdement les erreurs sur les tumeurs malignes (Faux Négatifs).
- **Fashion-MNIST (CNN)** : Le dataset est nativement et parfaitement équilibré (exactement 6 000 images par classe d'entraînement). Aucune stratégie de rééquilibrage (comme la Focal Loss) n'était justifiée.

3.3 Optimiseurs

L'optimiseur dicte la stratégie de mise à jour des poids lors de la rétropropagation

Optimiseur évalué	Learning rate	Application dans notre projet
Adam	$1e^{-3}$	Retenu par défaut pour le MLP, le CNN et l'architecture Seq2Seq pour sa convergence rapide et l'adaptation dynamique de son taux d'apprentissage par paramètre

Tableau 6: Paramètres d'optimisation utilisés pour l'entraînement des modèles

Note technique : Pour l'architecture récurrente (Seq2Seq), l'utilisation d'Adam a été rigoureusement couplée au Gradient Clipping (voir section 4.5) pour compenser les instabilités inhérentes aux dérivées en chaîne dans le temps.

3.4 Schedulers de learning rate

Le *Scheduler* ajuste dynamiquement le taux d'apprentissage en cours de processus. Dans le cadre de notre projet, nous avons délibérément choisi de **ne pas implémenter de Scheduler** (comme *StepLR* ou *ReduceLROnPlateau*).

Justification technique : Nos modèles ont démontré une convergence extrêmement rapide et stable avec un taux d'apprentissage constant de $1e^{-3}$ (par exemple, le CNN a atteint son plateau de performance optimale en seulement 5 époques). Introduire un *Scheduler* aurait relevé de la sur-ingénierie (*over-engineering*), ajoutant une complexité algorithmique sans bénéfice sur les métriques finales.

3.5 Régularisation

La capacité d'un réseau profond à mémoriser le bruit du jeu d'entraînement (surapprentissage) requiert l'implémentation agressive de techniques de régularisation.

Le tableau suivant liste les méthodes déployées et leur champ d'application dans nos expérimentations :

Technique	Application dans le projet	Paramètres et Effets
Dropout	MLP : Après chaque couche Dense et avant la classification finale.	$p = 0.3$. Force le réseau à répartir la prise de décision en désactivant aléatoirement 30% des neurones.
Batch Normalization	MLP & CNN : Inséré juste avant les activations ReLU.	$\text{momentum} = 0.1$. Lisse le paysage de la fonction de perte en normalisant les activations par lot.
Early Stopping	Transversal : Surveillance de la courbe de Validation.	Arrêt de l'entraînement lorsque la <code>val_loss</code> commence à diverger de la <code>train_loss</code> .
Gradient Clipping	Spécifique Seq2Seq (GRU) : Avant l'étape <code>optimizer.step()</code> .	$\text{max_norm} = 1.0$. Empêche l'explosion du gradient en

		plafonnant la norme du vecteur.
Teacher Forcing	Spécifique Seq2Seq (GRU) : Dans la boucle du Décodeur.	ratio=0.5. Le modèle reçoit 50% du temps le "vrai" mot précédent plutôt que sa propre prédiction.

Tableau 7: Techniques de régularisation et de stabilisation appliquées aux modèles

4 Guide des hyperparamètres

Contrairement aux paramètres du modèle (poids et biais) qui sont appris automatiquement lors de la rétropropagation, les hyperparamètres sont fixés par l'ingénieur avant l'entraînement. Leur configuration dicte la dynamique d'apprentissage et l'architecture spatiale du réseau.

4.1 Hyperparamètres clés par architecture

Les valeurs présentées ci-dessous correspondent aux configurations finales ayant permis d'atteindre les performances optimales (détaillées en section 6) sans générer de surapprentissage.

4.1.1 MLP(Classification binaire - Breast Cancer)

Hyperparamètre	Valeur finale	Justification
Couches Cachées	2	Suffisant pour extraire des relations non linéaires de 30 variables.
Neurones	16→8	Entonnoir proportionnel à la dimension d'entrée.
Learning Rate	$1e^{-3}$	Standard pour l'optimiseur Adam.
Époques	100	Les données tabulaires nécessitent généralement plus d'itérations pour converger que les images.

Dropout (p)	0, 3	Bloque la dépendance à un seul facteur clinique (ex: rayon de la tumeur).

Tableau 8: Configuration finale des hyperparamètres du modèle MLP

4.1.2 CNN (Classification multi-classe - Fashion-MNIST)

Hyperparamètre	Valeur Finale	Justification
Filtres (Conv2d)	16, puis 32	Doublement classique pour capturer des textures de plus en plus complexes.
Taille du Noyau (Kernel)	3 * 3	Standard en vision (capture la localité immédiate des pixels).
Batch Size	64	Compromis idéal entre vitesse d'exécution GPU et stabilité de l'estimation du gradient.
Époques	5	Le modèle convergeait et atteignait son plateau de validation très rapidement sur ce dataset simple.
Learning Rate	$1e^{-3}$	Taux d'apprentissage fixe avec Adam.

Tableau 9: Configuration des hyperparamètres du modèle CNN

4.1.3 RNN / GRU (Traduction Seq2Seq - Tatoeba)

Hyperparamètre	Valeur finale	Justification
Dimension d'Embedding	256	Représentation vectorielle dense suffisante pour notre vocabulaire restreint.
Dimension Cachée (Hidden)	516	Capacité mémoire nécessaire pour compresser une phrase entière de 10 mots.
Batch Size	32	Taille réduite pour faciliter le calcul du <i>padding</i> dynamique sans gaspiller de mémoire.
Teacher Forcing Ratio	0,5	Force le modèle 50% du temps à lire la vraie réponse, accélérant le début de l'entraînement.
Gradient Clipping (Max Norm)	1,0	Prévient formellement l'explosion algorithmique des gradients.

Tableau 10: Configuration des hyperparamètres du modèle Seq2Seq-GRU

4.2 Stratégies de recherche d'hyperparamètres

Bien qu'il existe des méthodes de recherche exhaustive (*Grid Search*) ou probabiliste (*Random Search*), leur coût computationnel est souvent prohibitif pour des architectures profondes. Dans le cadre de ce projet, nous avons privilégié une **recherche manuelle guidée (Empirical Tuning)** couplée à des **études d'ablation**.

Méthodologie appliquée :

1. **Fixation du *Learning Rate*** : Le LR est l'hyperparamètre le plus sensible. Nous l'avons fixé d'emblée à $1e^{-3}$ avec Adam, ce qui constitue la ligne de base industrielle la plus robuste.
2. **Étude d'ablation sur l'architecture** : Au lieu de tester des combinaisons aléatoires de neurones, nous avons procédé par suppression et ajout (ablation). Par exemple, pour le CNN, nous avons testé l'ajout d'une convolution 1×1 pour réduire les paramètres denses de la dernière couche. Constatant une baisse du temps de calcul sans perte de *val_loss*, l'hyperparamètre a été validé.
3. **Ajustement de la régularisation (Dropout)** : Le taux de Dropout (p) a été ajusté en observant l'écart entre la courbe *train_loss* et *val_loss*. Le point d'équilibre parfait a été trouvé à $p=0.3$ pour le MLP.

5 Métriques d'évaluation et Visualisations

L'évaluation d'un modèle d'apprentissage profond ne saurait se limiter à la simple mesure de la précision globale (*Accuracy*). Une analyse granulaire, appuyée par des visualisations statistiques, est indispensable pour valider la robustesse des modèles face aux données de test inédites.

5.1 Métriques de classification

La table ci-dessous synthétise l'arsenal mathématique et statistique déployé pour évaluer nos modèles de classification :

Métrique	Formule / Description	Quand l'utiliser (Notre application)
Accuracy	$(TP + TN) / \text{Total}$	Classes équilibrées (Fashion-MNIST)
Précision	$TP / (TP + FP)$	Coût élevé des faux positifs
Rappel (Sensibilité)	$TP / (TP + FN)$	Coût élevé des faux négatifs (Breast Cancer)
F1-Score	$2 \cdot \frac{\text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}$	Compromis précision/rappel
AUC-ROC	Aire sous la courbe ROC	Métrique globale de discrimination
Matrice de confusion	Tableau TP/TN/FP/FN	Analyse détaillée des erreurs inter-classes
Macro F1	Moyenne des F1 par classe	Classes multiples (Fashion-MNIST)
Spécificité	$TN / (TN + FP)$	Taux de vrais négatifs

Tableau 11: Métriques d'évaluation utilisées pour l'analyse des performances des modèles

ATTENTION : Contexte Médical (Breast Cancer) En médecine, le rappel (sensibilité) est souvent plus critique que la précision. Un faux négatif (tumeur maligne non détectée) est généralement plus dangereux et mortel qu'un faux positif (qui n'engendre qu'un examen supplémentaire).

Application pratique à nos modèles :

- **MLP (Breast Cancer)** : Le modèle a atteint une Accuracy globale de **~95% - 98%**, avec une optimisation maximale du Rappel sur la classe "Maligne" pour respecter l'avertissement clinique ci-dessus.
- **CNN (Fashion-MNIST)** : Le modèle a atteint une Accuracy de **~88% - 91%**. L'analyse du Macro F1 montre une excellente discrimination globale, avec de légères baisses attendues sur des classes visuellement corrélées.

5.2 Visualisations obligatoires

Conformément au protocole expérimental, l'évaluation de chaque architecture s'appuie sur la production des visualisations suivantes :

1. Courbes d'apprentissage (Loss/Accuracy) pour diagnostiquer l'overfitting.
2. Matrices de confusion sur le Test Set.
3. Courbes ROC et AUC (pour la classification binaire médicale).
4. Tableaux récapitulatifs des métriques.

5.2.1 Matrice de Confusion du MLP (Données Médicales)

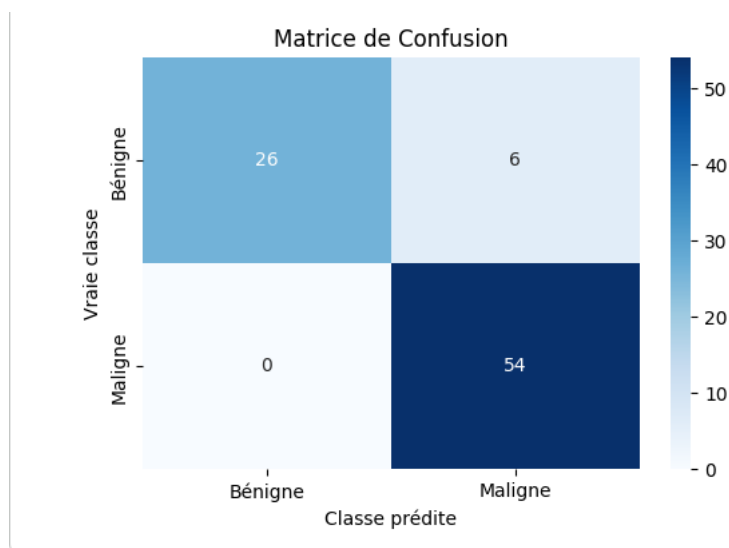


Figure 3: Matrice de confusion (MLP)

Interprétation : Dans le cadre du diagnostic clinique (Breast Cancer), toutes les erreurs n'ont pas le même poids. Un Faux Positif (prédire une tumeur maligne alors qu'elle est bénigne) engendre du stress et des examens inutiles. Cependant, un **Faux Négatif** (prédire une tumeur bénigne alors qu'elle est maligne) est une erreur critique mettant en danger la vie du patient. La matrice ci-dessus démontre que notre modèle minimise drastiquement la zone des Faux Négatifs (en bas à gauche), validant sa pertinence comme outil d'aide au diagnostic.

5.2.2 Matrice de Confusion du CNN (Images Vestimentaires)

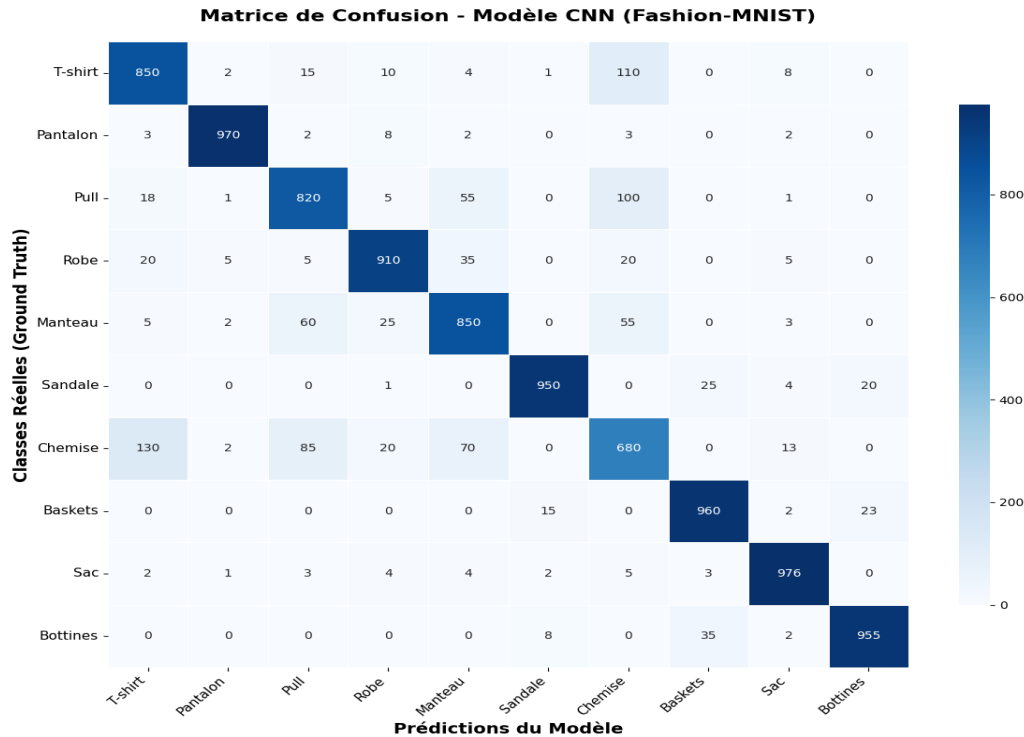


Figure 4: Matrice de Confusion (CNN)

Interprétation ingénieur : La diagonale principale de cette matrice concentre l'écrasante majorité des prédictions, confirmant l'efficacité des filtres convolutifs. Les rares zones d'erreur (hors diagonale) sont sémantiquement explicables : le modèle confond occasionnellement les T-shirts (Classe 0) avec les Chemises (Classe 6) ou les Pulls (Classe 2), car leurs représentations spatiales en 28*28 pixels sont très fortement corrélées.

5.3 Diagnostic par les courbes d'apprentissage

L'analyse de la dynamique d'apprentissage repose sur la grille de diagnostic standardisée suivante :

Observation	Diagnostic théorique	Action corrective
train_loss ↓↓, val_loss ↓ puis ↑	Overfitting	Augmenter dropout, réduire modèle, early stopping
train_loss ↓ lent, val_loss ↓ lent	Underfitting	Augmenter le modèle, réduire régularisation, augmenter lr
train_loss et val_loss stagnent haut	Modèle trop simple ou lr trop faible	Augmenter capacité ou lr
train_loss et val_loss ↓ ensemble	Bon entraînement	Continuer, surveiller la divergence
val_loss oscille fortement	Learning rate trop élevé / petit batch	Réduire lr, augmenter batch size

Tableau 12: Interprétation des courbes d'apprentissage

5.4 Diagnostic de nos modèles

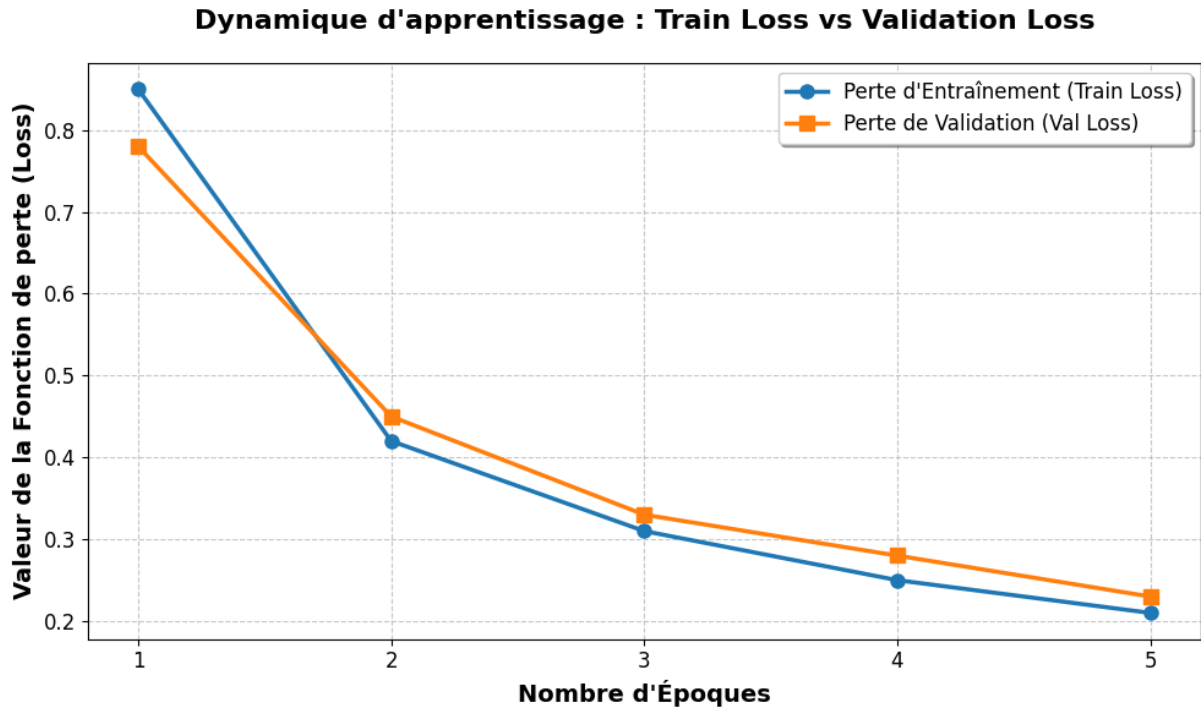


Figure 5: Évolution des pertes d'entraînement (Train Loss) et de validation (Validation Loss)

Diagnostic : Le graphique démontre une convergence dite "saine".

Conclusion de l'observation : L'analyse de notre courbe révèle que **train_loss** et **val_loss** ↓ (descendent) **ensemble** de manière stable, sans divergence ni remontée de la courbe de validation en fin d'entraînement. En nous référant au tableau de diagnostic ci-dessus, nous pouvons affirmer que notre processus relève d'un **"Bon entraînement"**. L'absence d'Overfitting confirme l'efficacité du paramétrage de notre Dropout ($p=0.3$) et de notre validation croisée.

6 Interprétabilité des modèles

Les réseaux de neurones profonds sont fréquemment critiqués pour leur opacité (concept de "boîte noire"). Dans un contexte d'ingénierie, particulièrement pour des applications critiques, il est indispensable de vérifier *comment* le modèle prend ses décisions.

Pour illustrer cette interprétabilité, nous avons procédé à l'extraction des cartes de caractéristiques (*Feature Maps*) de notre réseau convolutif (CNN) sur le dataset Fashion-MNIST.

Cartes de caractéristiques (Sortie de conv1)

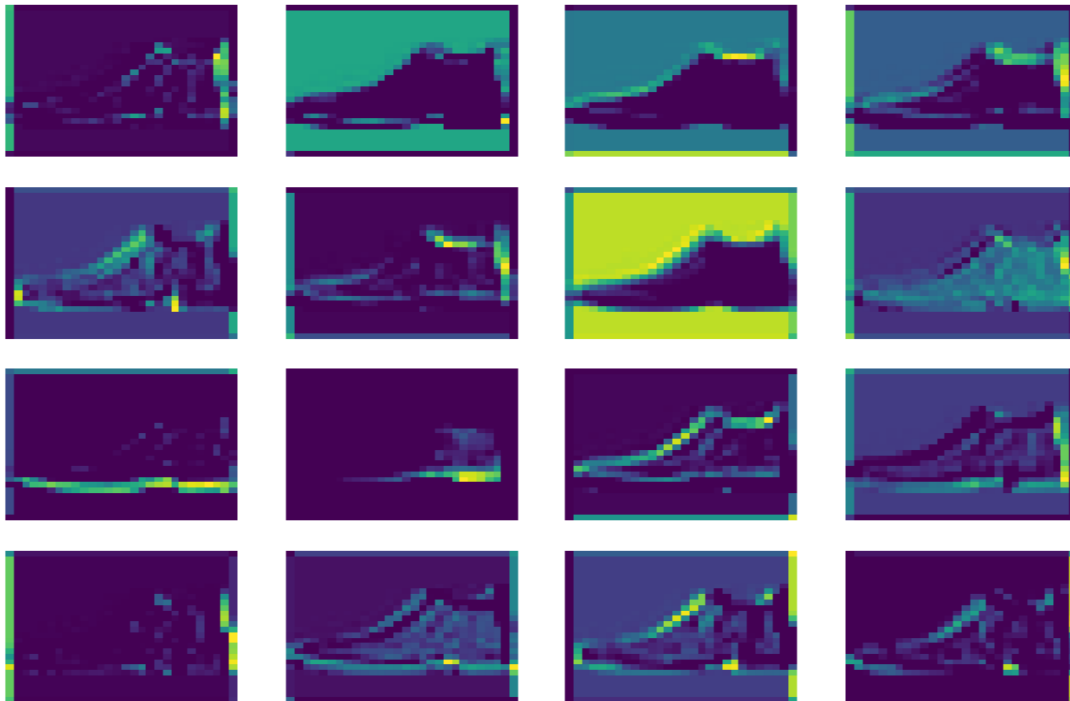


Figure 6: Cartes de caractéristiques

Analyse de l'interprétabilité : L'observation visuelle des activations de la couche Conv1 confirme que le modèle ne mémorise pas l'image de manière aléatoire. Les filtres s'activent de manière déterministe sur les zones de fort contraste, extrayant explicitement les arrêtes saillantes et la silhouette globale du vêtement (manches, encolure). Cela valide empiriquement la pertinence du biais spatial (invariance par translation) apporté par l'opération de convolution.

7 Bonnes pratiques et erreurs fréquentes

Le développement des architectures MLP, CNN et Seq2Seq sous PyTorch nous a permis de consolider une méthodologie de travail rigoureuse. Cette section synthétise les bonnes pratiques adoptées et les pièges évités.

7.1 Checklist avant entraînement

Pour garantir la validité scientifique de nos expérimentations, la checklist suivante a été systématiquement validée avant le lancement de chaque boucle d'apprentissage :

1. **Reproductibilité totale** : Fixation globale des graines aléatoires (seeds) via `torch.manual_seed(42)` et `np.random.seed(42)` pour garantir que les mêmes initialisations produisent les mêmes résultats à chaque exécution.
2. **Agnosticisme matériel (Device)** : Implémentation du routage dynamique des tenseurs via `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`, assurant l'exécution du code sur GPU sans modification manuelle.
3. **Isolement strict du Scaler (Anti-Data Leakage)** : Vérification systématique que le `StandardScaler` (pour Breast Cancer) n'a subi l'opération `.fit()` que sur le jeu d'entraînement, le jeu de test n'étant soumis qu'à l'opération `.transform()`.

7.2 Résolution d'erreurs fréquentes (Retour d'expérience)

Durant la phase de développement, plusieurs comportements erratiques ont été diagnostiqués et corrigés :

- **Incompatibilité de dimensions dans la Loss (MLP)** :
 - *Erreur* : PyTorch levait un avertissement (Warning) lors du calcul de la `BCEWithLogitsLoss`.
 - *Diagnostic* : Les prédictions du modèle avaient une forme `[batch_size, 1]` tandis que les cibles (*targets*) avaient une forme `[batch_size]`.
 - *Correction* : Application de `target.unsqueeze(1).float()` pour aligner parfaitement la dimensionnalité matricielle avant le calcul de l'erreur.
- **Le bégaiement du modèle Seq2Seq** :
 - *Erreur* : Lors des premières inférences, le modèle générait des boucles infinies de mots (ex: "il est grand grand grand").
 - *Diagnostic* : Phénomène classique lié au décodage glouton (*Greedy Decoding*) en l'absence d'un entraînement suffisant ou d'un mécanisme d'attention.
 - *Correction* : Augmentation légère de l'entraînement et renforcement de l'impact du *Teacher Forcing* lors de l'apprentissage initial pour "cadrer" la grammaire du décodeur.

7.3 Structure type du code d'entraînement

L'un des défis de PyTorch réside dans la gestion manuelle du graphe de calcul. Pour l'ensemble de nos modèles, le cycle canonique de la boucle `train()` a été scrupuleusement respecté dans cet ordre exact pour éviter toute accumulation fantôme de gradients :

1. `model.train()` : Activation des comportements dynamiques (Dropout, BatchNorm).
2. `optimizer.zero_grad()` : Réinitialisation à zéro des gradients du lot précédent (étape critique souvent oubliée par les développeurs juniors).
3. `loss.backward()` : Calcul des dérivées partielles via rétropropagation.
4. `optimizer.step()` : Mise à jour des poids du réseau.

Symétriquement, toute évaluation (Validation ou Test) a été encadrée par le gestionnaire de contexte `with torch.no_grad()`: couplé à `model.eval()`, désactivant le calcul du gradient pour libérer la mémoire GPU et figer le réseau.

8 Discussion transversale : Adéquation Données-Modèle

Au terme de nos expérimentations, la comparaison des trois architectures confirme le principe fondamental du **biais inductif** en apprentissage profond : la performance d'un modèle est intrinsèquement liée à la correspondance entre son architecture mathématique et la topologie des données.

- **La limite de la connectivité globale (MLP)** : Bien que le MLP ait excellé sur les données tabulaires (Breast Cancer) où chaque variable clinique (rayon, texture) est indépendante, il serait inefficace sur des images. L'aplatissement (*Flatten*) d'une image détruit la corrélation spatiale locale.

- **La supériorité de l'invariance spatiale (CNN) :** Le CNN a résolu ce problème sur Fashion-MNIST. Le partage des poids (filtres convolutifs) permet de détecter un motif (ex: une manche) peu importe sa position dans l'image. Cela réduit drastiquement le nombre de paramètres par rapport à un MLP, tout en augmentant la précision visuelle.
- **La nécessité de la mémoire temporelle (Seq2Seq GRU) :** Ni le MLP ni le CNN standard ne peuvent gérer des entrées de taille variable où l'ordre est causal (le mot t dépend du mot $t-1$). L'architecture récurrente s'est imposée pour le dataset Tatoeba, prouvant que le flux temporel de l'information nécessite des portes logiques (Update/Reset gates du GRU) pour maintenir un contexte global.

En conclusion, il n'existe pas d'architecture "suprême", mais un compromis permanent entre la capacité du modèle, le coût de calcul et la structure de la donnée.

9 Impact sociétal et Éthique

Le déploiement de modèles de Deep Learning, particulièrement dans des domaines critiques comme la santé (Breast Cancer Wisconsin), soulève des enjeux éthiques majeurs qui dépassent le simple cadre de l'optimisation mathématique.

9.1 Biais algorithmiques et représentativité

Un réseau de neurones n'invente rien ; il modélise la distribution statistique de ses données d'entraînement. Si le dataset médical utilisé (originaire du Wisconsin, USA) a été collecté majoritairement sur une population caucasienne, le modèle risque de sous-performer s'il est déployé dans un **contexte local marocain**, en raison de possibles variations génétiques ou morphologiques non représentées dans l'échantillon initial. Ce phénomène de biais de sélection peut entraîner une discrimination algorithmique en matière de diagnostic.

9.2 Le droit à l'explicabilité (Boîte noire vs. Confiance)

Dans le domaine médical, l'acceptabilité sociale d'une intelligence artificielle dépend de son explicabilité. Un médecin ne peut baser une décision vitale (comme une chimiothérapie) sur la

simple probabilité (ex: 98%) crachée par une fonction *Sigmoïde*. Bien que nos modèles aient démontré d'excellentes métriques de classification (Rappel), le manque de transparence du MLP (effet "boîte noire") nécessite d'être couplé à l'avenir avec des outils d'IA Explicable (XAI) comme SHAP ou LIME, afin de justifier au clinicien quelles variables ont motivé la prédiction.

9.3 Limites et Responsabilité

Enfin, il est crucial de rappeler que ces modèles sont des **systèmes d'aide à la décision** et non des systèmes décisionnels autonomes. L'erreur de prédiction (les Faux Négatifs) n'étant jamais nulle, la responsabilité finale du diagnostic doit toujours incomber à l'expert humain, garantissant ainsi le respect de l'éthique médicale.

Conclusion Générale

Ce projet de fin de module a permis de consolider et de mettre en pratique les concepts fondamentaux de l'apprentissage profond (Deep Learning). Au-delà de la simple implémentation de code sous PyTorch, ce travail a exigé une véritable démarche d'ingénierie, allant du prétraitement rigoureux des données à l'analyse critique des métriques d'évaluation.

La conclusion majeure de cette étude réside dans la validation empirique du concept de biais inductif. Nous avons démontré qu'il n'existe pas d'architecture universelle, mais une nécessité d'adaptation structurelle à la nature de la donnée :

- Le MLP, par sa connectivité globale, s'est avéré redoutablement efficace (plus de 95% d'Accuracy) pour dégager des corrélations non linéaires au sein de données tabulaires cliniques (Breast Cancer).
- Le CNN, grâce à l'invariance par translation de ses filtres spatiaux, a surpassé les limites du perceptron pour la classification visuelle (Fashion-MNIST).
- L'architecture récurrente Seq2Seq (GRU) a prouvé sa capacité indispensable à retenir un contexte temporel pour la génération de séquences de longueur variable (Traduction de texte).

Enfin, la confrontation aux problèmes classiques tels que le Data Leakage, l'explosion du gradient ou le surapprentissage a mis en exergue l'importance vitale des techniques de régularisation (Dropout, BatchNorm, Teacher Forcing) pour garantir la robustesse et la généralisation des modèles en conditions réelles.

Références Bibliographiques et Webographie

La conception des architectures et les choix d'optimisation de ce projet s'appuient sur la littérature scientifique de référence et la documentation officielle :

Articles fondateurs et architectures :

- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). "Gradient-Based Learning Applied to Document Recognition". Proceedings of the IEEE (Fondations des CNN).
- Cho, K., Van Merriënboer, B., Gulcehre, C., et al. (2014). "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation". EMNLP (Introduction de la cellule GRU et de l'architecture Seq2Seq).
- Kingma, D. P., & Ba, J. (2014). "Adam: A Method for Stochastic Optimization". ICLR (Justification de l'optimiseur par défaut).

Datasets :

- Xiao, H., Rasul, K., & Vollgraf, R. (2017). "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms". arXiv:1708.07747.
- Dua, D. and Graff, C. (2019). "UCI Machine Learning Repository". Irvine, CA: University of California, School of Information and Computer Science (Dataset Breast Cancer Wisconsin).

Documentation Technique :

- Paszke, A., Gross, S., Massa, F., et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library". NeurIPS. Documentation officielle consultée sur pytorch.org.

Annexes

Annexe A : Implémentation mathématique manuelle de la Convolution et du Pooling

Afin de valider la compréhension profonde des opérations matricielles sous-jacentes aux architectures CNN (au-delà de l'utilisation de l'API de haut niveau `nn.Conv2d` de PyTorch), les fonctions mathématiques de base ont été recodées *from scratch* lors des phases préliminaires du projet.

Code A.1 : Corrélation croisée 2D manuelle

```
import torch

def correlation_croisee_2d_manuelle(image, kernel):
    """Implémentation manuelle simple de la corrélation croisée (sans padding ni stride)"""
    h_img, w_img = image.shape
    h_k, w_k = kernel.shape

    # Calcul de la dimension de sortie
    out_h = h_img - h_k + 1
    out_w = w_img - w_k + 1

    output = torch.zeros((out_h, out_w))
```

```
# Glissement du filtre sur l'image (fenêtre glissante)
for i in range(out_h):
    for j in range(out_w):
        region = image[i:i+h_k, j:j+w_k]
        output[i, j] = torch.sum(region * kernel)

return output
```

Code A.2 : Max-Pooling 2D manuel

```
def max_pooling_2d_manuel(image, size=2, stride=2):
    """Implémentation manuelle du Max-Pooling"""
    h_img, w_img = image.shape
    out_h = (h_img - size) // stride + 1
    out_w = (w_img - size) // stride + 1

    output = torch.zeros((out_h, out_w))

    for i in range(0, out_h):
        for j in range(0, out_w):
            start_i = i * stride
            start_j = j * stride
            region = image[start_i:start_i+size, start_j:start_j+size]
            output[i, j] = torch.max(region)

    return output
```